MLOps Assignment 01

Heart Disease Classification

End-to-End Machine Learning, Experiment Tracking, CI/CD and Deployment

Submission detail	Information
Course	Machine Learning Operations (MLOps) — AIMLCZG523
Assignment	Assignment 01 — Experimental Learning Assignment
Student	Shaba Joseph
Student ID	2025AE05285
Submission date	20 October 2026
Repository	https://github.com/shabajoseph-git/MLOPS

Academic project report

Contents

1. Executive Summary
2. Project Objectives and Scope
3. Environment and Setup
4. Data Exploration and Preparation
5. Model Development and Evaluation
6. Experiment Tracking with MLflow
7. System Architecture
8. Testing and Quality Checks
9. CI/CD and Deployment
10. Monitoring and Operational Readiness
11. Reproducibility and Submission Checklist
12. Limitations and Conclusion

Appendix A. Useful Commands

Appendix B. Evidence to Include

Before submission: verify every metric and runtime status against the final project outputs. Replace evidence placeholders with genuine screenshots from your own run.

1. Executive Summary

This project implements an end-to-end MLOps workflow for a heart-disease classification task. It brings together data exploration, model training and evaluation, experiment tracking, automated testing, an API service, containerisation and Kubernetes deployment. The aim is to make the machine-learning workflow reproducible, testable and deployable rather than stopping at a notebook or a single trained model.

The project uses Python-based source code and MLflow to record training runs. A FastAPI service exposes the trained model for inference and a health endpoint reports whether the model has loaded. The development history recorded the following evidence; re-check these values immediately before submitting because local state can change.

Evidence item	Recorded result	Final verification
Automated tests	3 passed	Run pytest -q again
API health endpoint	HTTP 200; {"status":"ok","model_loaded":"true"}	Call /health while service is running
MLflow experiment	heart-disease-classification	Confirm in MLflow UI and local mlruns/
Experiment ID	952052864617791423	Confirm against the current tracking store
Runs observed	12 finished and 4 failed at the time checked	Refresh MLflow and explain failed runs if relevant
Kubernetes workload	heart-disease-api; pod observed 1/1 Running	Run kubectl get deployments,pods

The results above are a snapshot from development, not a guarantee that the service or cluster is currently running.

2. Project Objectives and Scope

- Build a repeatable data preparation and model-training workflow.
- Compare candidate classifiers using cross-validation and held-out test metrics.
- Track parameters, metrics and model artefacts with MLflow.
- Add automated tests and code-quality checks to reduce regressions.
- Expose model inference through an API with a health endpoint.
- Package the application and deploy it to a local Kubernetes environment.
- Document commands, evidence and known limitations for reproducibility.

3. Environment and Setup

The project was developed on Windows using PowerShell and a Python virtual environment. The working project directory is:

```
E:\SHABA\MLOPS\mlops_heart_disease_starter
```

3.1 Prerequisites

Component	Purpose
Python 3.11 and venv	Run project scripts in an isolated environment
Git	Version control and repository history
Docker Desktop	Build and run the container image

Minikube	Local Kubernetes cluster
kubectl	Inspect and manage Kubernetes resources
Pandoc (optional)	Convert Markdown documentation into Word/PDF
MiKTeX/XeLaTeX (PDF only)	PDF generation when using Pandoc

3.2 Python dependencies

The project dependency set includes the following packages or package groups; use the repository's pinned requirements file as the source of truth.

- ucimlrepo for data access where used; pandas and NumPy for data handling.
- scikit-learn for preprocessing, model fitting and evaluation.
- matplotlib and seaborn for exploratory visualisation.
- MLflow and joblib for experiment tracking and model artefacts.
- FastAPI, Uvicorn and Pydantic for the inference API.
- pytest and httpx for automated tests; Ruff for linting.
- prometheus-client if metrics instrumentation is enabled.

3.3 Environment activation and installation

```
cd E:\SHABA\MLOPS\mlops_heart_disease_starter\n.\.venv\Scripts\Activate.ps1\npython --
version\npython -m pip install -r requirements.txt
```

Use the exact requirements filename present in the repository. If dependencies are managed through pyproject.toml or another lock file, follow that project configuration instead.

4. Data Exploration and Preparation

The workflow uses a heart-disease dataset referenced in the project as data/heart_disease.csv. Verify that the final repository contains the data file or the documented retrieval/preparation script, and that any data redistribution restrictions are respected.

4.1 Exploration checklist

- Inspect the dataset shape, column names and data types.
- Check missing values, duplicate rows and target-label distribution.
- Review summary statistics and plausible feature ranges.
- Visualise target balance and selected feature relationships.
- Document any cleaning, encoding, scaling or train/test splitting decisions.

4.2 Expected evidence

Suggested output	Purpose
artifacts/eda/target_distribution.png	Show class balance
artifacts/eda/correlation_heatmap.png	Summarise numeric feature relationships
artifacts/eda/missing_values.png or table	Document missingness checks
artifacts/eda/summary_statistics.csv	Keep reproducible descriptive statistics

These are suggested evidence paths. Include only files that your code actually generated; do not create screenshots or outputs that imply an analysis was performed if it was not.

5. Model Development and Evaluation

Two candidate models were compared: Logistic Regression and Random Forest. Cross-validation was used during model selection, and test-set metrics were recorded to describe held-out performance. Confirm the metric values from the final training run or MLflow before submission.

Metric	Logistic Regression	Random Forest
Mean CV accuracy	0.855187	0.801616
Test accuracy	0.868852	0.885246
Test precision	0.812500	0.818182
Test recall	0.928571	0.964286
Test ROC-AUC	0.966450	0.946429

5.1 Model selection

The recorded selection decision favoured Logistic Regression based on the mean 5-fold cross-validation ROC-AUC: 0.902478 compared with 0.898511 for Random Forest. Random Forest had higher recorded test accuracy and recall, while Logistic Regression had higher recorded test ROC-AUC. The report should clearly distinguish the metric used for model selection from metrics reported on the held-out test set.

5.2 Interpretation

- Accuracy is the fraction of predictions that are correct.
- Precision measures the proportion of predicted positive cases that are truly positive.
- Recall measures the proportion of actual positive cases detected by the classifier.
- ROC-AUC measures ranking discrimination across thresholds; it does not establish clinical usefulness.

Because this is a health-related classification exercise, results are for academic demonstration only and must not be used to diagnose or guide treatment.

6. Experiment Tracking with MLflow

MLflow records the training experiment, run parameters, evaluation metrics and model artefacts. The experiment name observed during development was heart-disease-classification, with experiment ID 952052864617791423 and a local mlruns/ tracking directory.

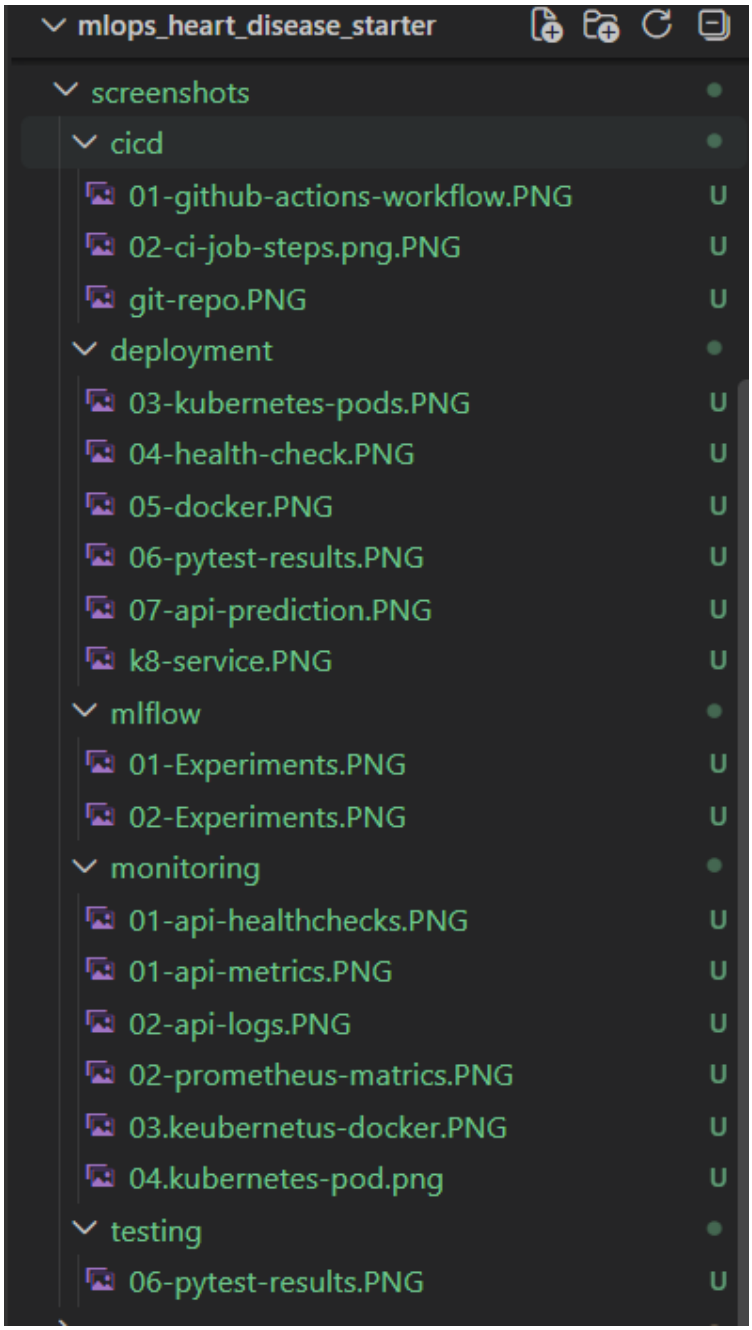
6.1 What to verify

- The selected experiment is visible in the MLflow UI.
- Successful runs contain the parameters and metrics used for comparison.
- The chosen model artefact is present and can be loaded by the inference service.
- Failed runs are understood; avoid hiding failures if they reveal a genuine workflow issue.
- The report's final metrics agree with the logged run and training output.

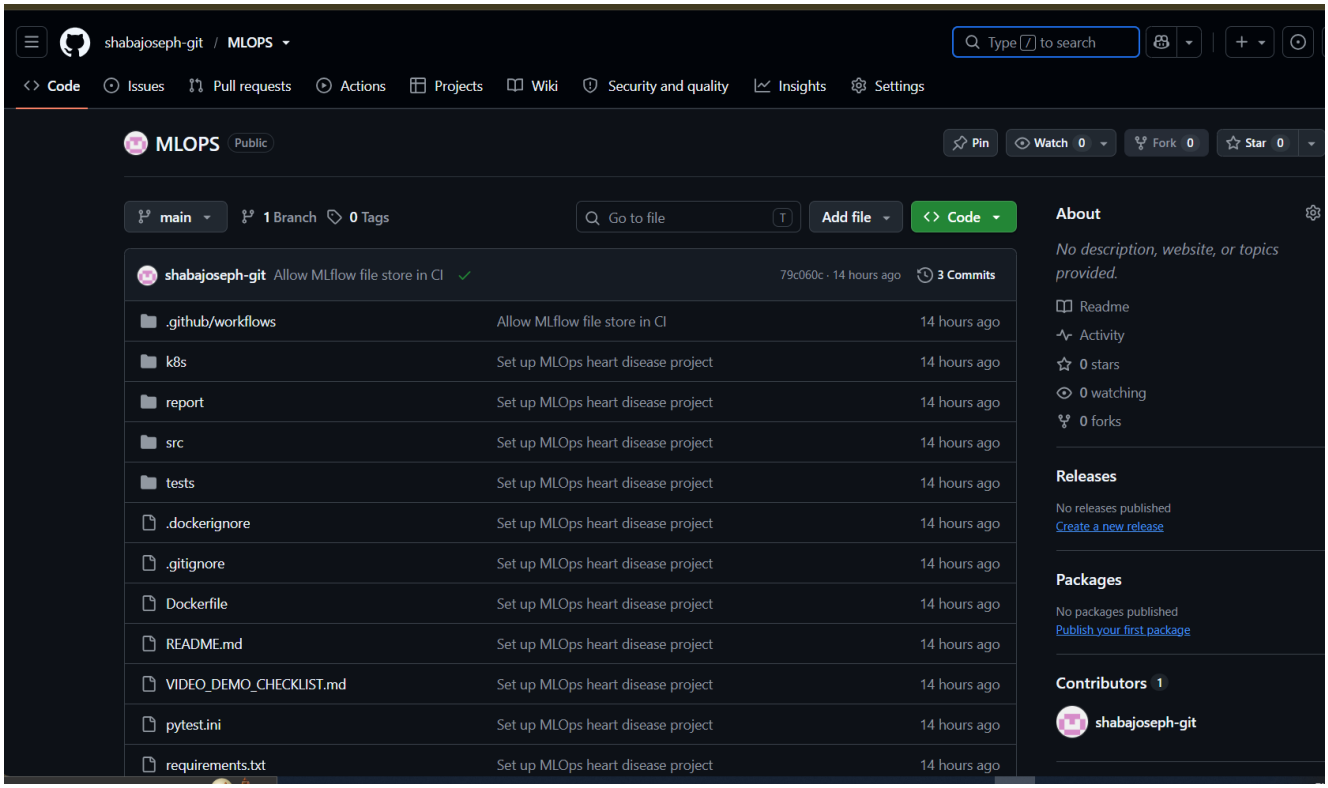
6.2 Screenshot evidence

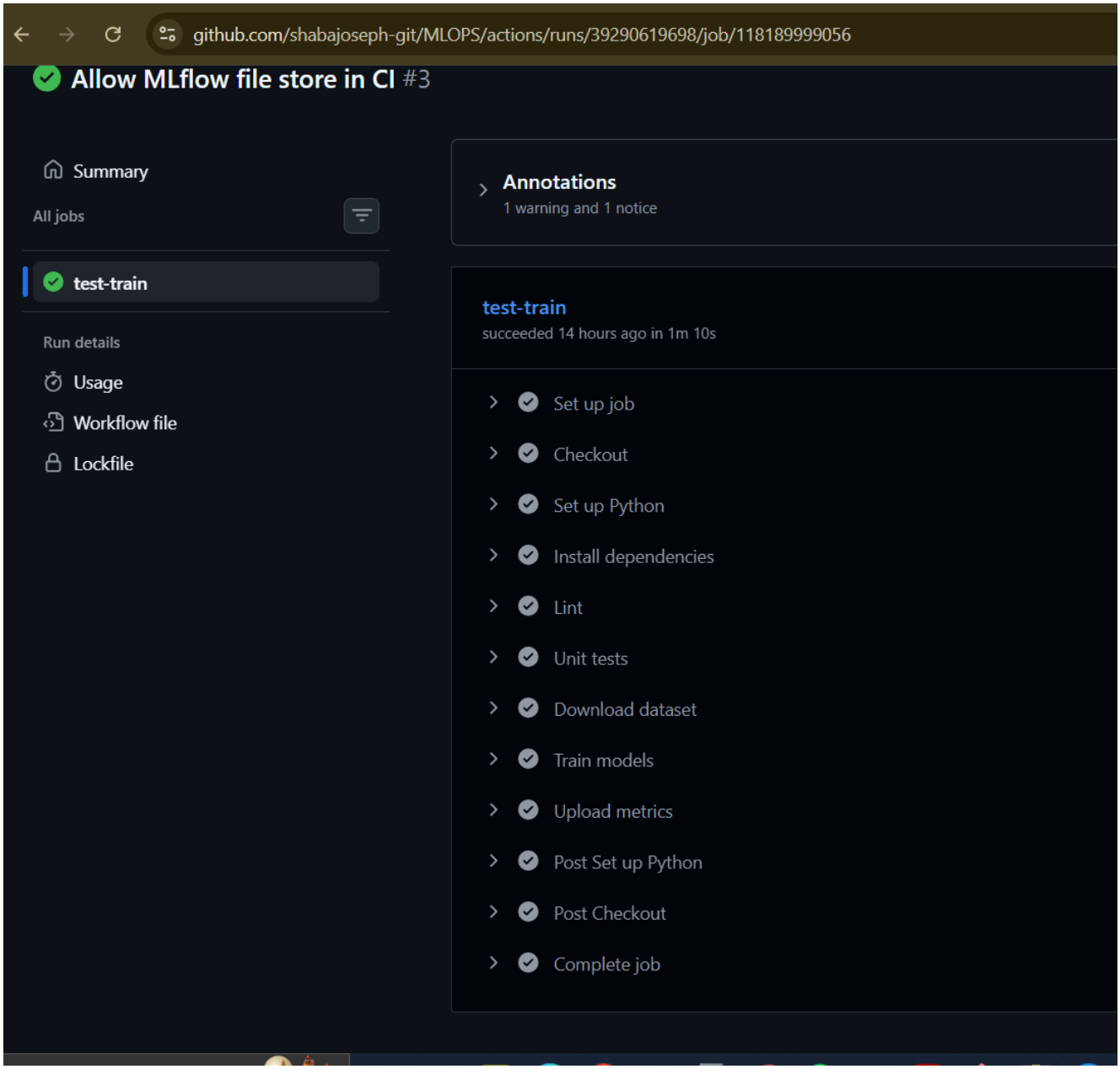
- Experiment overview showing the experiment name and runs.
- Run comparison showing model names and key metrics.
- Selected run showing parameters, metrics and logged model artefact.

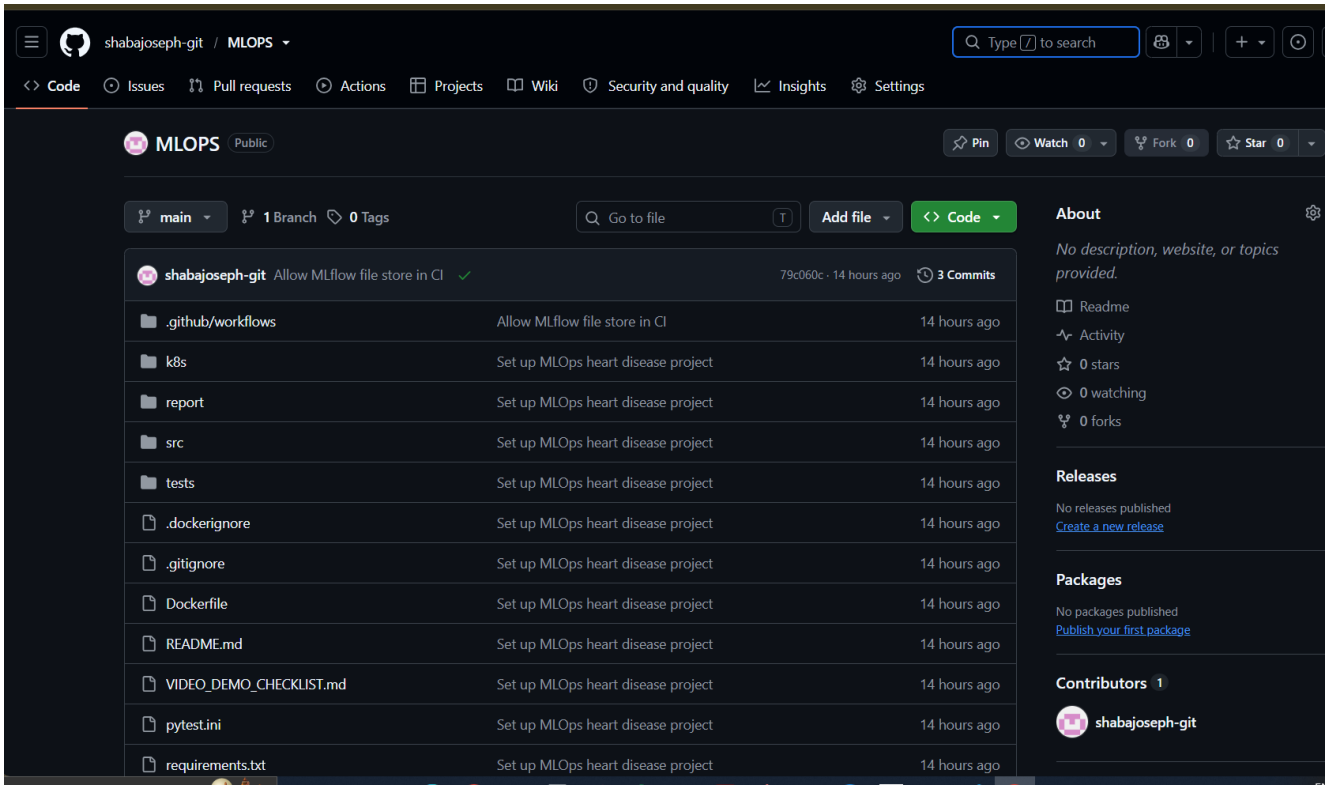
##Vs Scode Screenshot path



CI-CD


shabajoseph-git / MLOPS
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security and quality
Insights
Settings
MLOPS
Public
Pin
Watch 0
Fork 0
Star 0
main
1 Branch
0 Tags
Go to file
Add file
Code
shabajoseph-git Allow MLflow file store in CI
79c060c · 14 hours ago
3 Commits
.github/workflows Allow MLflow file store in CI 14 hours ago
k8s Set up MLOps heart disease project 14 hours ago
report Set up MLOps heart disease project 14 hours ago
src Set up MLOps heart disease project 14 hours ago
tests Set up MLOps heart disease project 14 hours ago
.dockerignore Set up MLOps heart disease project 14 hours ago
.gitignore Set up MLOps heart disease project 14 hours ago
Dockerfile Set up MLOps heart disease project 14 hours ago
README.md Set up MLOps heart disease project 14 hours ago
VIDEO_DEMO_CHECKLIST.md Set up MLOps heart disease project 14 hours ago
pytest.ini Set up MLOps heart disease project 14 hours ago
requirements.txt Set up MLOps heart disease project 14 hours ago
About
No description, website, or topics provided.
Readme
Activity
0 stars
0 watching
0 forks
Releases
No releases published
Create a new release
Packages
No packages published
Publish your first package
Contributors 1
shabajoseph-git


github.com/shabajoseph-git/MLOPS/actions/runs/39290619698/job/118189999056
Allow MLflow file store in CI #3
Summary
All jobs
test-train
Run details
Usage
Workflow file
Lockfile
Annotations
1 warning and 1 notice
test-train
succeeded 14 hours ago in 1m 10s
Set up job
Checkout
Set up Python
Install dependencies
Lint
Unit tests
Download dataset
Train models
Upload metrics
Post Set up Python
Post Checkout
Complete job



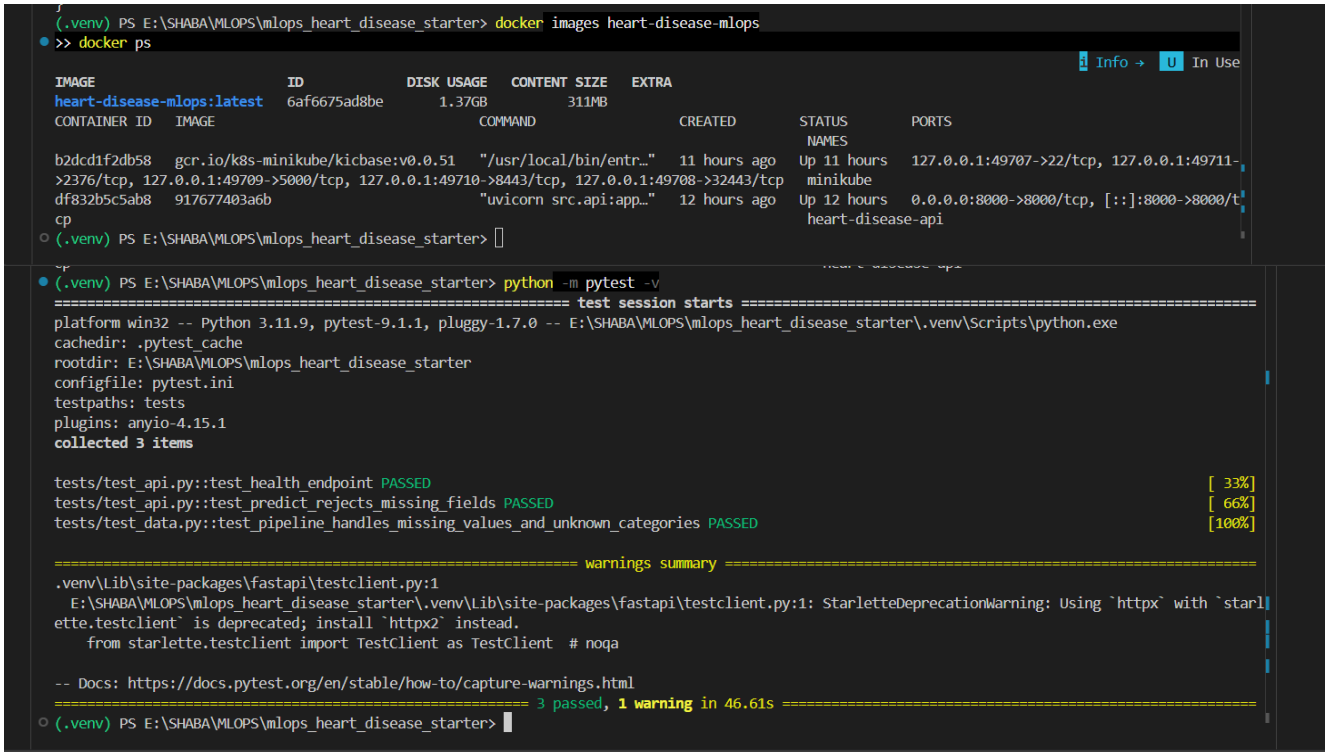
Deployment

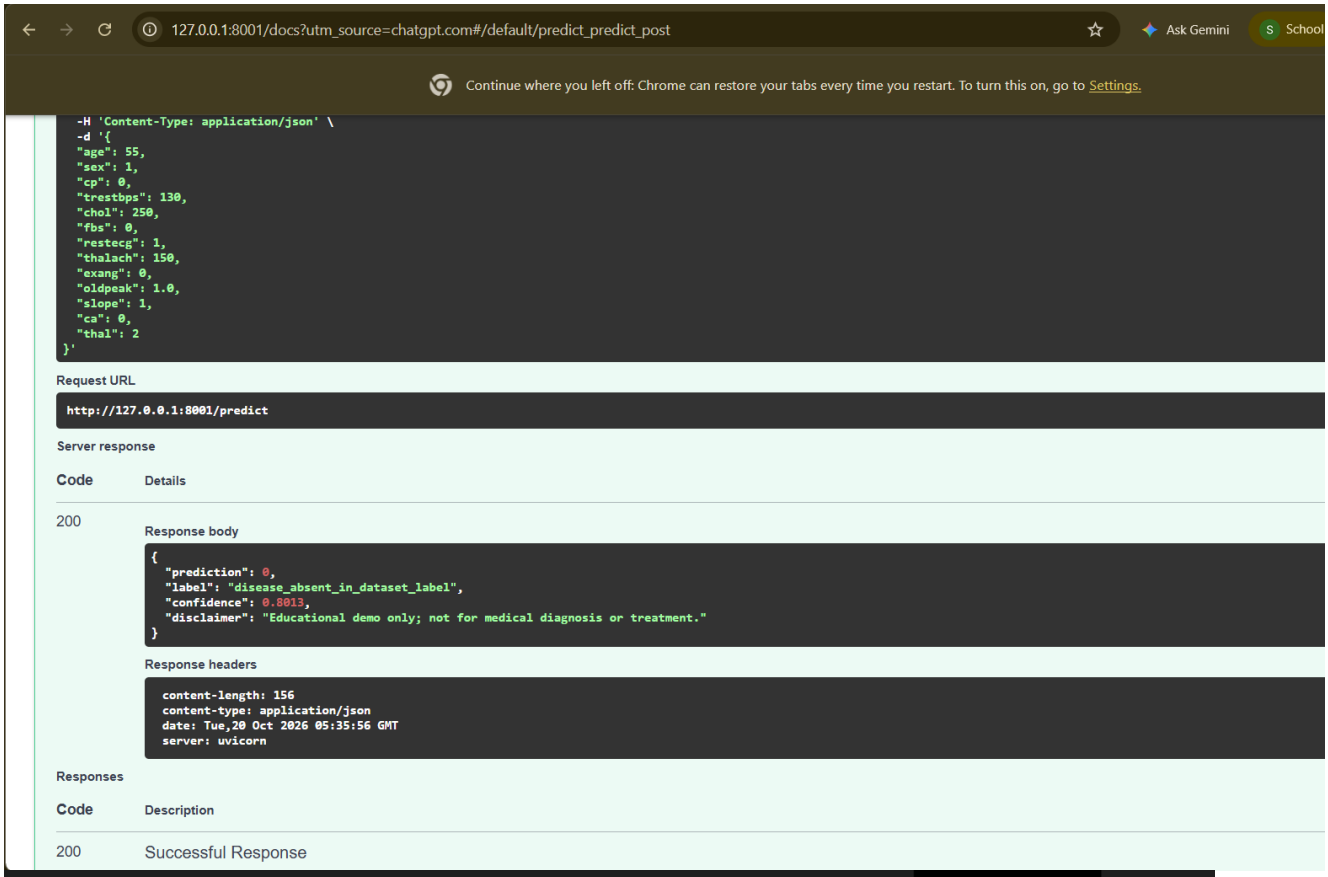
```
(.venv) PS E:\SHABA\MLOPS\mlops_heart_disease_starter> kubectl get deployments,pods,services
NAME                                 READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/heart-disease-api    1/1     1            1           10h

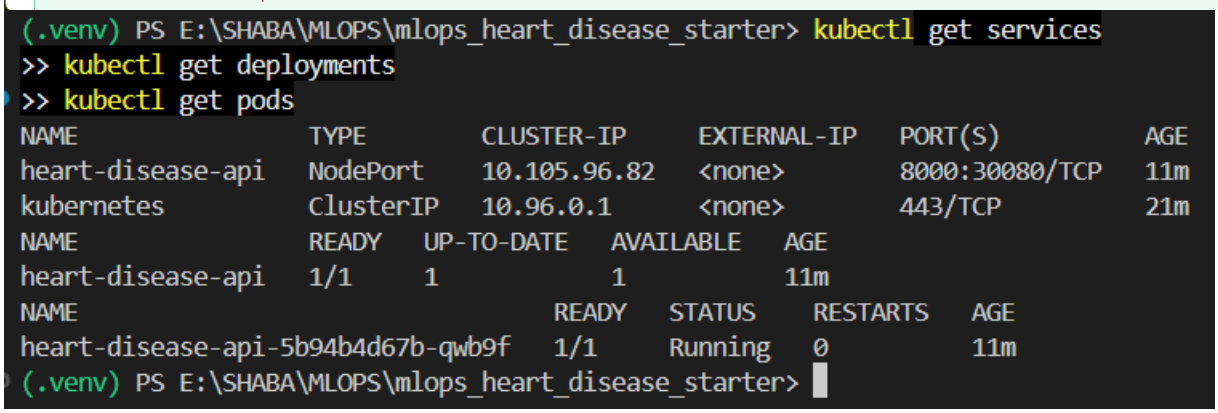
NAME                                     READY   STATUS    RESTARTS   AGE
pod/heart-disease-api-56d68bcd44-66bfs   1/1     Running   0          40m

NAME                        TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)          AGE
service/heart-disease-api   NodePort    10.105.96.82   <none>        8000:30080/TCP   10h
service/kubernetes          ClusterIP   10.96.0.1      <none>        443/TCP          11h
(.venv) PS E:\SHABA\MLOPS\mlops_heart_disease_starter>
```

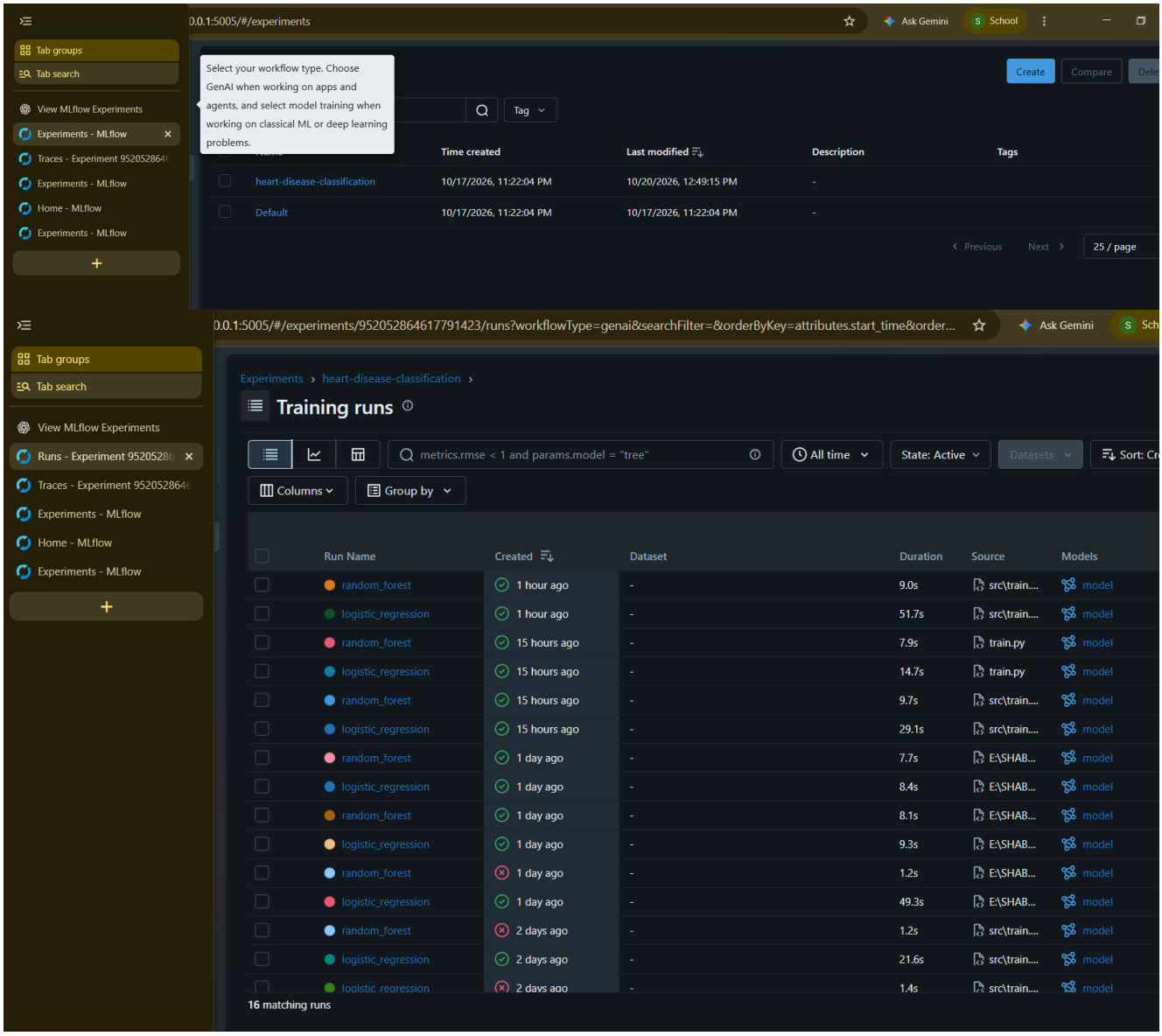
```
(.venv) PS E:\SHABA\MLOPS\mlops_heart_disease_starter> Invoke-RestMethod http://127.0.0.1:8081/health | ConvertTo-Json
{
    "status":  "ok",
    "model_loaded":  "true"
}
(.venv) PS E:\SHABA\MLOPS\mlops_heart_disease_starter>
```


(.venv) PS E:\SHABA\MLOPS\mlops_heart_disease_starter> docker images heart-disease-mlops
>> docker ps
Info
In Use
IMAGE ID DISK USAGE CONTENT SIZE EXTRA
heart-disease-mlops:latest 6af6675ad8be 1.37GB 311MB
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
b2dcd1f2db58 gcr.io/k8s-minikube/kicbase:v0.0.51 "/usr/local/bin/entr…" 11 hours ago Up 11 hours 127.0.0.1:49707->22/tcp, 127.0.0.1:49711->2376/tcp, 127.0.0.1:49709->5000/tcp, 127.0.0.1:49710->8443/tcp, 127.0.0.1:49708->32443/tcp minikube
df832b5c5ab8 917677403a6b "uvicorn src.api:app…" 12 hours ago Up 12 hours 0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp heart-disease-api
(.venv) PS E:\SHABA\MLOPS\mlops_heart_disease_starter>
(.venv) PS E:\SHABA\MLOPS\mlops_heart_disease_starter> python -m pytest -v
test session starts
platform win32 -- Python 3.11.9, pytest-9.1.1, pluggy-1.7.0 -- E:\SHABA\MLOPS\mlops_heart_disease_starter\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: E:\SHABA\MLOPS\mlops_heart_disease_starter
configfile: pytest.ini
testpaths: tests
plugins: anyio-4.15.1
collected 3 items
tests/test_api.py::test_health_endpoint PASSED [33%]
tests/test_api.py::test_predict_rejects_missing_fields PASSED [66%]
tests/test_data.py::test_pipeline_handles_missing_values_and_unknown_categories PASSED [100%]
warnings summary
.venv\Lib\site-packages\fastapi\testclient.py:1
E:\SHABA\MLOPS\mlops_heart_disease_starter\.venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
from starlette.testclient import TestClient # noqa
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
3 passed, 1 warning in 46.61s
(.venv) PS E:\SHABA\MLOPS\mlops_heart_disease_starter>


127.0.0.1:8001/docs?utm_source=chatgpt.com#/default/predict_predict_post
Ask Gemini
School
Continue where you left off: Chrome can restore your tabs every time you restart. To turn this on, go to Settings.
-H 'Content-Type: application/json' \
-d '{
"age": 55,
"sex": 1,
"cp": 0,
"trestbps": 130,
"chol": 250,
"fbs": 0,
"restecg": 1,
"thalach": 150,
"exang": 0,
"oldpeak": 1.0,
"slope": 1,
"ca": 0,
"thal": 2
}'
Request URL
http://127.0.0.1:8001/predict
Server response
Code
Details
200
Response body
{
"prediction": 0,
"label": "disease_absent_in_dataset_label",
"confidence": 0.8013,
"disclaimer": "Educational demo only; not for medical diagnosis or treatment."
}
Response headers
content-length: 156
content-type: application/json
date: Tue,20 Oct 2026 05:35:56 GMT
server: uvicorn
Responses
Code
Description
200
Successful Response


(.venv) PS E:\SHABA\MLOPS\mlops_heart_disease_starter> kubectl get services
>> kubectl get deployments
>> kubectl get pods
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
heart-disease-api NodePort 10.105.96.82 <none> 8000:30080/TCP 11m
kubernetes ClusterIP 10.96.0.1 <none> 443/TCP 21m
NAME READY UP-TO-DATE AVAILABLE AGE
heart-disease-api 1/1 1 1 11m
NAME READY STATUS RESTARTS AGE
heart-disease-api-5b94b4d67b-qwb9f 1/1 Running 0 11m
(.venv) PS E:\SHABA\MLOPS\mlops_heart_disease_starter>

MLFLOW

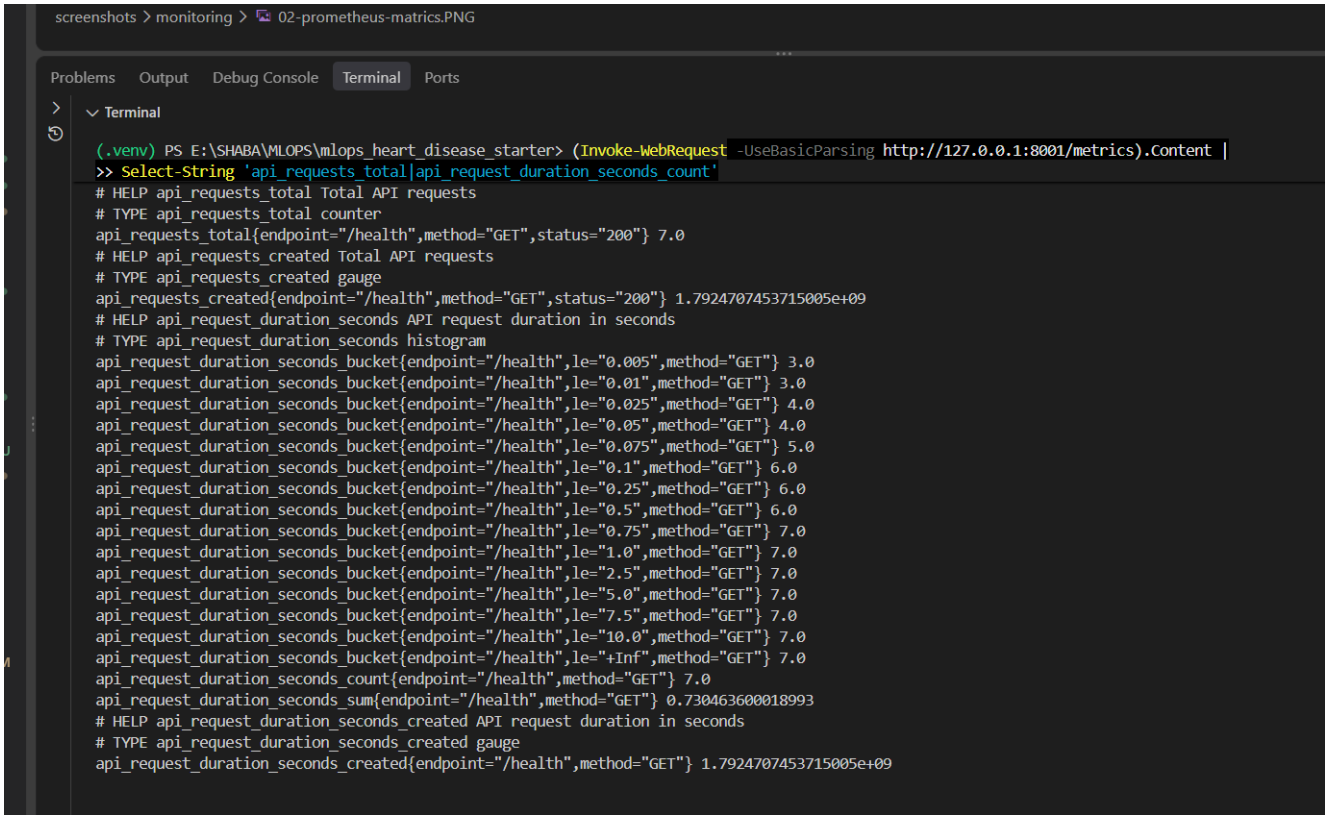

0.0.1:5005/#/experiments
Ask Gemini
School
Tab groups
Tab search
View MLflow Experiments
Experiments - MLflow
Traces - Experiment 952052864
Experiments - MLflow
Home - MLflow
Experiments - MLflow
Select your workflow type. Choose GenAI when working on apps and agents, and select model training when working on classical ML or deep learning problems.
Create
Compare
Tag
Time created
Last modified
Description
Tags
heart-disease-classification
10/17/2026, 11:22:04 PM
10/20/2026, 12:49:15 PM
Default
10/17/2026, 11:22:04 PM
10/17/2026, 11:22:04 PM
Previous
Next
25 / page
0.0.1:5005/#/experiments/952052864617791423/runs?workflowType=genai&searchFilter=&orderByKey=attributes.start_time&order...
Ask Gemini
Tab groups
Tab search
View MLflow Experiments
Runs - Experiment 9520528
Traces - Experiment 952052864
Experiments - MLflow
Home - MLflow
Experiments - MLflow
Experiments › heart-disease-classification ›
Training runs
metrics.rmse < 1 and params.model = "tree"
All time
State: Active
Datasets
Columns
Group by
Run Name
Created
Dataset
Duration
Source
Models
random_forest
1 hour ago
9.0s
src\train....
model
logistic_regression
1 hour ago
51.7s
src\train....
model
random_forest
15 hours ago
7.9s
train.py
model
logistic_regression
15 hours ago
14.7s
train.py
model
random_forest
15 hours ago
9.7s
src\train....
model
logistic_regression
15 hours ago
29.1s
src\train....
model
random_forest
1 day ago
7.7s
E:\SHAB...
model
logistic_regression
1 day ago
8.4s
E:\SHAB...
model
random_forest
1 day ago
8.1s
E:\SHAB...
model
logistic_regression
1 day ago
9.3s
E:\SHAB...
model
random_forest
1 day ago
1.2s
E:\SHAB...
model
logistic_regression
1 day ago
49.3s
E:\SHAB...
model
random_forest
2 days ago
1.2s
src\train....
model
logistic_regression
2 days ago
21.6s
src\train....
model
logistic_regression
2 days ago
1.4s
src\train....
model
16 matching runs

Monitoring

```
Terminal

PS E:\SHABA\MLOPS\mlops_heart_disease_starter> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& e
:\SHABA\MLOPS\mlops_heart_disease_starter\.venv\Scripts\Activate.ps1)
(.venv) PS E:\SHABA\MLOPS\mlops_heart_disease_starter> Write-Host "`n========== 1. API HEALTH CHECK ==========" -Foregro
undColor Cyan
>> try {
>>     Invoke-RestMethod http://127.0.0.1:8001/health |
>>         ConvertTo-Json -Compress
>> } catch {
>>     Write-Host "Local API unavailable on port 8001"
>> }
>>
>> Write-Host "`n========== 2. PROMETHEUS METRICS ==========" -ForegroundColor Cyan
>> try {
>>     $metrics = (Invoke-WebRequest -UseBasicParsing `
>>         http://127.0.0.1:8001/metrics).Content
>>
>>     $metrics | Select-String `
>>         'api_requests_total|api_request_duration_seconds_count'
>> } catch {
>>     Write-Host "Metrics unavailable on port 8001"
>> }
>>
>> Write-Host "`n========== 3. KUBERNETES STATUS ==========" -ForegroundColor Cyan
>> kubectl get deployments,pods,services
>>
>> Write-Host "`n========== 4. DOCKER IMAGE ==========" -ForegroundColor Cyan
>> docker images heart-disease-mlops
>>
>> Write-Host "`n========== CHECKS COMPLETED ==========" -ForegroundColor Green

========== 1. API HEALTH CHECK ==========
{"status":"ok","model_loaded":"true"}
```



```
heart-disease-api
(.venv) PS E:\SHABA\MLOPS\mlops_heart_disease_starter> uvicorn src.api:app --host 127.0.0.1 --port 8001
INFO:     Started server process [22444]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:8001 (Press CTRL+C to quit)
INFO:     127.0.0.1:56477 - "GET /metrics HTTP/1.1" 307 Temporary Redirect
INFO:     127.0.0.1:56477 - "GET /metrics/ HTTP/1.1" 200 OK
2026-10-20 10:02:25,371 INFO method=GET path=/health status=200 duration_seconds=0.0995
INFO:     127.0.0.1:56494 - "GET /health HTTP/1.1" 200 OK
2026-10-20 10:07:32,329 INFO method=GET path=/health status=200 duration_seconds=0.0546
INFO:     127.0.0.1:56550 - "GET /health HTTP/1.1" 200 OK
2026-10-20 10:07:32,785 INFO method=GET path=/health status=200 duration_seconds=0.0016
INFO:     127.0.0.1:56550 - "GET /health HTTP/1.1" 200 OK
2026-10-20 10:07:32,821 INFO method=GET path=/health status=200 duration_seconds=0.0011
INFO:     127.0.0.1:56550 - "GET /health HTTP/1.1" 200 OK
INFO:     127.0.0.1:56559 - "GET /metrics HTTP/1.1" 307 Temporary Redirect
INFO:     127.0.0.1:56559 - "GET /metrics/ HTTP/1.1" 200 OK
2026-10-20 10:31:52,117 INFO method=GET path=/health status=200 duration_seconds=0.0154
INFO:     127.0.0.1:56892 - "GET /health HTTP/1.1" 200 OK
INFO:     127.0.0.1:56892 - "GET /metrics HTTP/1.1" 307 Temporary Redirect
INFO:     127.0.0.1:56892 - "GET /metrics/ HTTP/1.1" 200 OK
2026-10-20 10:46:08,167 INFO method=GET path=/health status=200 duration_seconds=0.5561
INFO:     127.0.0.1:57041 - "GET /health HTTP/1.1" 200 OK
2026-10-20 10:46:21,446 INFO method=GET path=/health status=200 duration_seconds=0.0021
INFO:     127.0.0.1:57060 - "GET /health HTTP/1.1" 200 OK
INFO:     127.0.0.1:57168 - "GET /metrics HTTP/1.1" 307 Temporary Redirect
INFO:     127.0.0.1:57168 - "GET /metrics/ HTTP/1.1" 200 OK
```

```
(.venv) PS E:\SHABA\MLOPS\mlops_heart_disease_starter> Write-Host "`n========== 1. API HEALTH CHECK ==========" -Foregro
undColor Cyan
>> try {
>>     Invoke-RestMethod http://127.0.0.1:8001/health |
>>         ConvertTo-Json -Compress
```

```
========== 2. PROMETHEUS METRICS ==========

# HELP python_gc_objects_collected_total Objects collected during gc
# TYPE python_gc_objects_collected_total counter
python_gc_objects_collected_total{generation="0"} 1094.0
python_gc_objects_collected_total{generation="1"} 76.0
python_gc_objects_collected_total{generation="2"} 5.0
# HELP python_gc_objects_uncollectable_total Uncollectable objects found during GC
# TYPE python_gc_objects_uncollectable_total counter
python_gc_objects_uncollectable_total{generation="0"} 0.0
python_gc_objects_uncollectable_total{generation="1"} 0.0
python_gc_objects_uncollectable_total{generation="2"} 0.0
# HELP python_gc_collections_total Number of times this generation was collected
# TYPE python_gc_collections_total counter
python_gc_collections_total{generation="0"} 253.0
python_gc_collections_total{generation="1"} 22.0
python_gc_collections_total{generation="2"} 2.0
# HELP python_info Python platform information
# TYPE python_info gauge
python_info{implementation="CPython",major="3",minor="11",patchlevel="9",version="3.11.9"} 1.0
# HELP api_requests_total Total API requests
# TYPE api_requests_total counter
api_requests_total{endpoint="/health",method="GET",status="200"} 5.0
# HELP api_requests_created Total API requests
# TYPE api_requests_created gauge
api_requests_created{endpoint="/health",method="GET",status="200"} 1.7924707453715005e+09
# HELP api_request_duration_seconds API request duration in seconds
# TYPE api_request_duration_seconds histogram
api_request_duration_seconds_bucket{endpoint="/health",le="0.005",method="GET"} 2.0
api_request_duration_seconds_bucket{endpoint="/health",le="0.01",method="GET"} 2.0
api_request_duration_seconds_bucket{endpoint="/health",le="0.025",method="GET"} 3.0
api_request_duration_seconds_bucket{endpoint="/health",le="0.05",method="GET"} 3.0
api_request_duration_seconds_bucket{endpoint="/health",le="0.075",method="GET"} 4.0
api_request_duration_seconds_bucket{endpoint="/health",le="0.1",method="GET"} 5.0
api_request_duration_seconds_bucket{endpoint="/health",le="0.25",method="GET"} 5.0
api_request_duration_seconds_bucket{endpoint="/health",le="0.5",method="GET"} 5.0
api_request_duration_seconds_bucket{endpoint="/health",le="0.75",method="GET"} 5.0
api_request_duration_seconds_bucket{endpoint="/health",le="1.0",method="GET"} 5.0
api_request_duration_seconds_bucket{endpoint="/health",le="2.5",method="GET"} 5.0
```

```
(.venv) PS E:\SHABA\MLOPS\mlops_heart_disease_starter> Write-Host "`n========== 1. API HEALTH CHECK ==========" -Foregro
undColor Cyan
>> try {
>>     Invoke-RestMethod http://127.0.0.1:8001/health |
>>         ConvertTo-Json -Compress
```

```
========== 2. PROMETHEUS METRICS ==========

# HELP python_gc_objects_collected_total Objects collected during gc
# TYPE python_gc_objects_collected_total counter
python_gc_objects_collected_total{generation="0"} 1094.0
python_gc_objects_collected_total{generation="1"} 76.0
python_gc_objects_collected_total{generation="2"} 5.0
# HELP python_gc_objects_uncollectable_total Uncollectable objects found during GC
# TYPE python_gc_objects_uncollectable_total counter
python_gc_objects_uncollectable_total{generation="0"} 0.0
python_gc_objects_uncollectable_total{generation="1"} 0.0
python_gc_objects_uncollectable_total{generation="2"} 0.0
# HELP python_gc_collections_total Number of times this generation was collected
# TYPE python_gc_collections_total counter
python_gc_collections_total{generation="0"} 253.0
python_gc_collections_total{generation="1"} 22.0
python_gc_collections_total{generation="2"} 2.0
# HELP python_info Python platform information
# TYPE python_info gauge
python_info{implementation="CPython",major="3",minor="11",patchlevel="9",version="3.11.9"} 1.0
# HELP api_requests_total Total API requests
# TYPE api_requests_total counter
api_requests_total{endpoint="/health",method="GET",status="200"} 5.0
# HELP api_requests_created Total API requests
# TYPE api_requests_created gauge
api_requests_created{endpoint="/health",method="GET",status="200"} 1.7924707453715005e+09
# HELP api_request_duration_seconds API request duration in seconds
# TYPE api_request_duration_seconds histogram
api_request_duration_seconds_bucket{endpoint="/health",le="0.005",method="GET"} 2.0
api_request_duration_seconds_bucket{endpoint="/health",le="0.01",method="GET"} 2.0
api_request_duration_seconds_bucket{endpoint="/health",le="0.025",method="GET"} 3.0
api_request_duration_seconds_bucket{endpoint="/health",le="0.05",method="GET"} 3.0
api_request_duration_seconds_bucket{endpoint="/health",le="0.075",method="GET"} 4.0
api_request_duration_seconds_bucket{endpoint="/health",le="0.1",method="GET"} 5.0
api_request_duration_seconds_bucket{endpoint="/health",le="0.25",method="GET"} 5.0
api_request_duration_seconds_bucket{endpoint="/health",le="0.5",method="GET"} 5.0
api_request_duration_seconds_bucket{endpoint="/health",le="0.75",method="GET"} 5.0
api_request_duration_seconds_bucket{endpoint="/health",le="1.0",method="GET"} 5.0
api_request_duration_seconds_bucket{endpoint="/health",le="2.5",method="GET"} 5.0
```

```
========== 3. KUBERNETES STATUS ==========
NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/heart-disease-api   1/1     1            1           10h

NAME                                    READY   STATUS    RESTARTS   AGE
pod/heart-disease-api-56d68bcd44-66bfs  1/1     Running   0          15m

NAME                        TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)          AGE
service/heart-disease-api   NodePort    10.105.96.82   <none>        8000:30080/TCP   10h
service/kubernetes          ClusterIP   10.96.0.1      <none>        443/TCP          10h

========== 4. DOCKER IMAGE ==========
```

i Info → U In Use

```
IMAGE                       ID             DISK USAGE   CONTENT SIZE   EXTRA
heart-disease-mlops:latest  6af6675ad8be       1.37GB          311MB

           CHECKS COMPLETED
```


```
========== 3. KUBERNETES STATUS ==========
NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/heart-disease-api   1/1     1            1           10h

NAME                                    READY   STATUS    RESTARTS   AGE
pod/heart-disease-api-56d68bcd44-66bfs  1/1     Running   0          15m

NAME                        TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)          AGE
service/heart-disease-api   NodePort    10.105.96.82   <none>        8000:30080/TCP   10h
service/kubernetes          ClusterIP   10.96.0.1      <none>        443/TCP          10h

========== 4. DOCKER IMAGE ==========
```

i Info → U In Use

```
IMAGE                       ID             DISK USAGE   CONTENT SIZE   EXTRA
heart-disease-mlops:latest  6af6675ad8be       1.37GB          311MB

           CHECKS COMPLETED
```

Testing

```
(.venv) PS E:\SHABA\MLOPS\mlops_heart_disease_starter> python -m pytest -v
============================= test session starts ==============================
platform win32 -- Python 3.11.9, pytest-9.1.1, pluggy-1.7.0 -- E:\SHABA\MLOPS\mlops_heart_disease_starter\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: E:\SHABA\MLOPS\mlops_heart_disease_starter
configfile: pytest.ini
testpaths: tests
plugins: anyio-4.15.1
collected 3 items

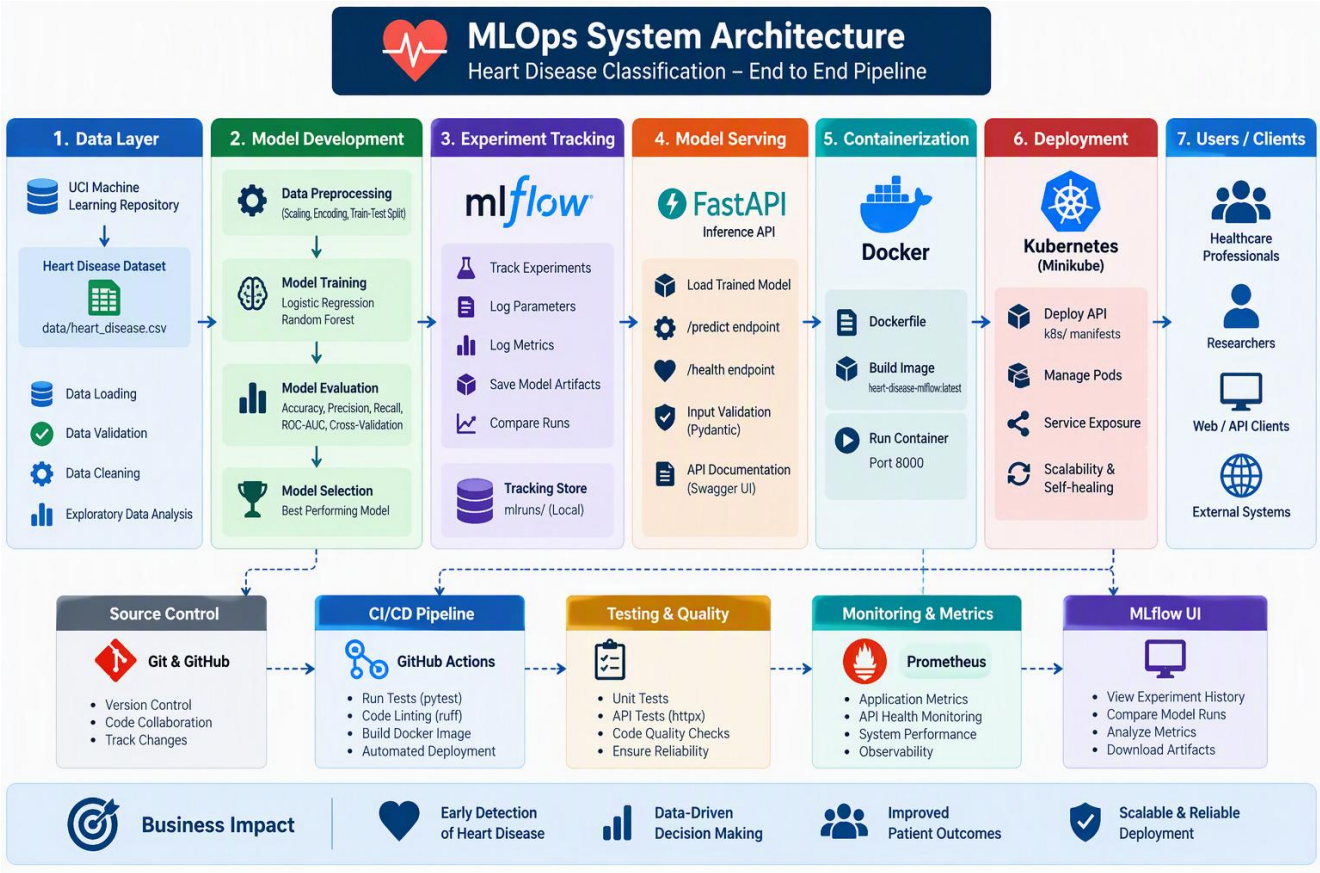
tests/test_api.py::test_health_endpoint PASSED                           [ 33%]
tests/test_api.py::test_predict_rejects_missing_fields PASSED            [ 66%]
tests/test_data.py::test_pipeline_handles_missing_values_and_unknown_categories PASSED [100%]

============================== warnings summary ===============================
.venv\Lib\site-packages\fastapi\testclient.py:1
  E:\SHABA\MLOPS\mlops_heart_disease_starter\.venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starl
ette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
======================== 3 passed, 1 warning in 46.61s ========================
(.venv) PS E:\SHABA\MLOPS\mlops_heart_disease_starter>
```

7. System Architecture

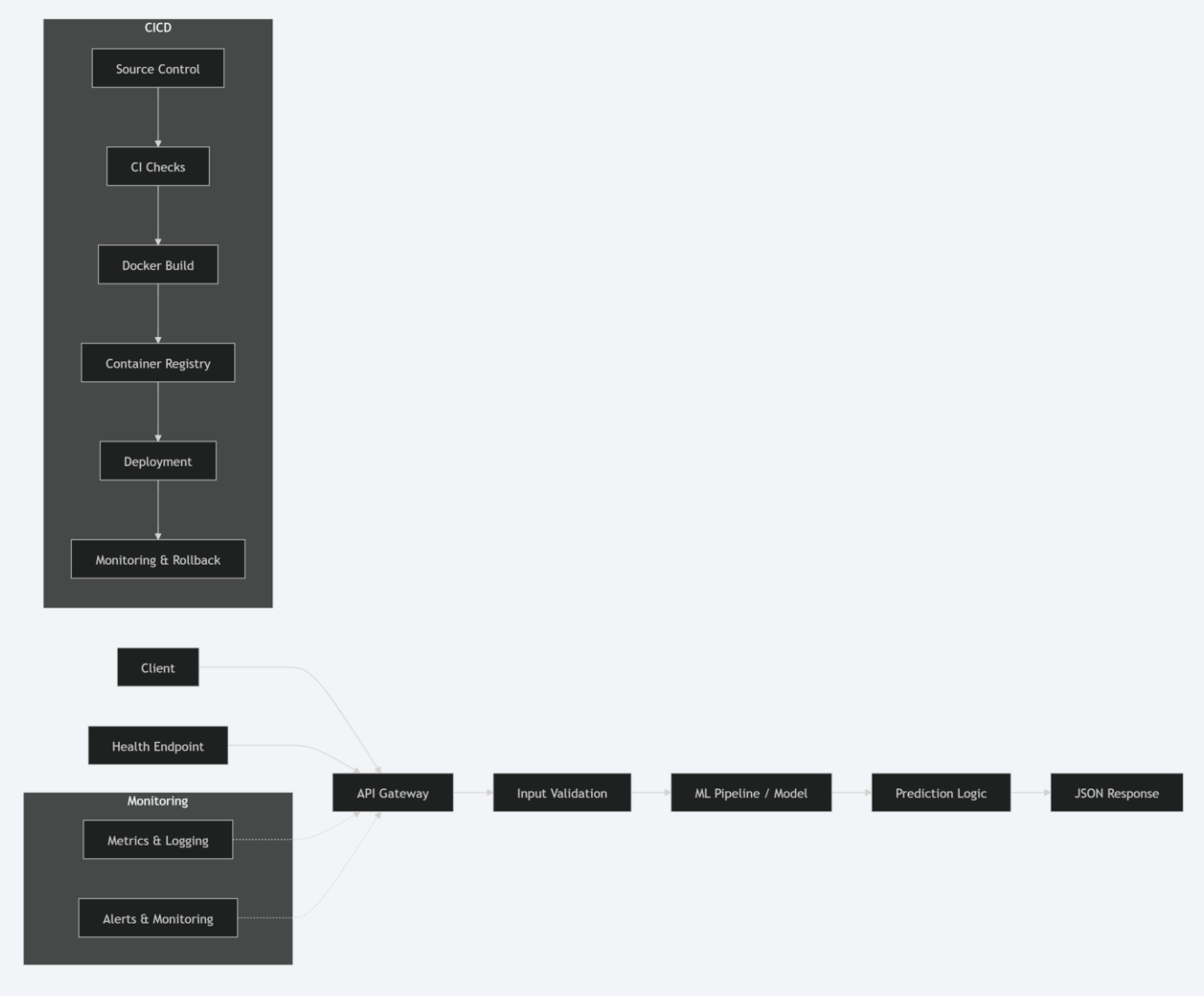
The workflow can be represented as a sequence from source data through training and experiment tracking to a model-serving API, packaged container and Kubernetes workload. Automated tests and CI checks provide quality gates around code changes.



Stage	Responsibility	Evidence
Data and preprocessing	Load, validate, split and transform data	Dataset/script and EDA outputs
Training and evaluation	Fit candidate models and compute metrics	Training logs and metric table

MLflow	Store run metadata and model artefacts	Experiment/run screenshots
API	Load the selected model and serve predictions	Health response and prediction test
Container	Package runtime and dependencies	Successful image build
Kubernetes	Run and expose the API workload	Deployment and pod status
CI/CD	Automate linting and tests on changes	Successful workflow run

7.1 Request flow



8. Testing and Quality Checks

The recorded test result was 3 passed. Re-run the test suite from the repository root before final submission and capture the terminal output. Tests should cover the core data/model logic and API behaviour appropriate to the project.

```
python -m pytest -q
python -m ruff check .
```

- Unit tests pass in a clean project environment.
- Linting completes without unresolved errors, or any exceptions are documented.
- The model can be loaded from the artefact path used by the API.
- The API returns a healthy status when the model is loaded.
- A representative prediction request returns the expected response structure.

9. CI/CD and Deployment

9.1 Continuous integration

The CI workflow should install dependencies and run code-quality checks and tests for changes pushed to the repository or submitted through a pull request. The final report should include a screenshot of a successful workflow run, plus the workflow file in the repository.

9.2 Containerisation

The application should be packaged with a Dockerfile and an appropriate .dockerignore file. Build and run commands depend on the image name and port configured in the repository; use those exact values rather than copying generic placeholders.

```
docker --version
docker build -t heart-disease-api .
```

9.3 Kubernetes deployment

The recorded Kubernetes workload name was heart-disease-api. Verify the active cluster, deployment and pods, and confirm that the service can be reached from the host using the access method configured in the manifests.

```
kubectl get deployments
kubectl get pods
kubectl get services
kubectl describe deployment heart-disease-api
```

9.4 API health verification

```
curl http://localhost:8000/health
```

The recorded response was {"status":"ok","model_loaded":"true"} with HTTP 200. Capture a fresh response after starting the final deployed service and ensure the model-loaded field is true.

10. Monitoring and Operational Readiness

If metrics instrumentation is included, document the metrics endpoint and show that it is reachable. At minimum, describe how to inspect application health, container logs and Kubernetes workload status. Monitoring screenshots should be included only if the corresponding feature is implemented and running.

- Health check confirms the application process and model-loading state.
- Container and pod logs can be inspected for startup or inference errors.
- Resource requests/limits and replica count are appropriate for the assignment scope, if configured.

- Model/data drift monitoring is not implied unless implemented and supported by evidence.

11. Reproducibility and Submission Checklist

- README documents setup, training, testing, API execution and deployment.
- Source code, configuration and manifests are committed to the repository.
- Dependency versions are recorded in the project dependency file.
- Training and evaluation commands run from the documented working directory.
- Final metrics match MLflow and the report.
- Screenshots are genuine, readable and organised by topic.
- No virtual environment, secrets, credentials or unnecessary large generated files are committed.
- Repository URL is accessible to the assessor.
- Any external dataset use and limitations are documented.

12. Limitations and Conclusion

This assignment demonstrates the main building blocks of an MLOps workflow: model experimentation, experiment tracking, testing, API serving, containerisation and Kubernetes deployment. The development record includes a passing test run and a successful health check, but all runtime states and reported metrics should be refreshed before submission.

The project is a learning exercise, not a validated medical device. Dataset representativeness, bias, calibration, external validation, threshold selection and clinical impact are outside the scope of a basic assignment unless separately evaluated. No result in this report should be used for medical diagnosis or treatment decisions.

Appendix A. Useful Commands

Task	Command / action
Activate virtual environment	.\.venv\Scripts\Activate.ps1
Check Python	python --version
Run tests	python -m pytest -q
Run lint	python -m ruff check .
Check MLflow runs	Open the configured MLflow UI and select heart-disease-classification
Check API health	curl http://localhost:8000/health
Check Kubernetes	kubectl get deployments,pods,services
Check Git status	git status --short

Appendix B. Evidence to Include

Folder	Recommended screenshot/evidence
screenshots/mlflow/	Experiment list, run comparison, selected run details and model artefact
screenshots/cicd/	Successful CI workflow and test/lint job details
screenshots/deployment/	Deployment and pod status, service details, API health response
screenshots/monitoring/	Metrics endpoint or monitoring view, only if implemented

artifacts/eda/	Actual plots/tables generated by the project